

```
[1]: import numpy as np
import pandas as pd
```

```
[2]: df = pd.read_csv('/home/subramanyaprasad/Machine Learning/Data Analysis/Notes/03-Pandas/tips.csv')
```

```
[3]: df
```

	total_bill	tip	sex	smoker	day	time	size	price_per_person	Payer Name	CC Number	Payment ID
0	16.99	1.01	Female	No	Sun	Dinner	2	8.49	Christy Cunningham	3560325168603410	Sun2959
1	10.34	1.66	Male	No	Sun	Dinner	3	3.45	Douglas Tucker	4478071379779230	Sun4608
2	21.01	3.50	Male	No	Sun	Dinner	3	7.00	Travis Walters	6011812112971322	Sun4458
3	23.68	3.31	Male	No	Sun	Dinner	2	11.84	Nathaniel Harris	4676137647685994	Sun5260
4	24.59	3.61	Female	No	Sun	Dinner	4	6.15	Tonya Carter	4832732618637221	Sun2251
...	...	...	...	...	...	...	...	...	...	...	...
239	29.03	5.92	Male	No	Sat	Dinner	3	9.68	Michael Avila	5296068606052842	Sat2657
240	2718	2.00	Female	Yes	Sat	Dinner	2	13.59	Monica Sanders	3506806155565404	Sat1766
241	22.67	2.00	Male	Yes	Sat	Dinner	2	11.34	Keith Wong	6011891618747196	Sat3880
242	17.82	1.75	Male	No	Sat	Dinner	2	8.91	Dennis Dixon	4375220550950	Sat17
243	18.78	3.00	Female	No	Thur	Dinner	2	9.39	Michelle Hardin	3511451626698139	Thur672

244 rows x 11 columns

```
[4]: def simple(num):
      return num*2
```

```
[5]: simple(2)
```

[5]: 4

Now, to convert a function to lambda expression:

```
[6]: lambda num : num*2
```

```
[6]: <function __main__.<lambda>(num)>
[6]: lambda num : num*2
```

```
[6]: <function __main__.<lambda>(num)>
```

```
[7]: df['total_bill'].apply(simple)
```

0	33.98
1	20.68
2	42.02
3	47.36
4	49.18
...	...
239	58.06
240	54.36
241	45.34
242	35.64
243	37.56

Name: total_bill, Length: 244, dtype: float64

```
[8]: df['total_bill'].apply(lambda num : num*2)
```

0	33.98
1	20.68
2	42.02
3	47.36
4	49.18
...	...
239	58.06
240	54.36
241	45.34
242	35.64
243	37.56

Name: total_bill, Length: 244, dtype: float64

There are various ways to use apply on multiple columns, but let's do the simplest one:

```
[9]: df.head()
```

	total_bill	tip	sex	smoker	day	time	size	price_per_person	Payer Name	CC Number	Payment ID
0	16.99	1.01	Female	No	Sun	Dinner	2	8.49	Christy Cunningham	3560325168603410	Sun2959
1	10.34	1.66	Male	No	Sun	Dinner	3	3.45	Douglas Tucker	4478071379779230	Sun4608
2	21.01	3.50	Male	No	Sun	Dinner	3	7.00	Travis Walters	6011812112971322	Sun4458
3	23.68	3.31	Male	No	Sun	Dinner	2	11.84	Nathaniel Harris	4676137647685994	Sun5260
4	24.59	3.61	Female	No	Sun	Dinner	4	6.15	Tonya Carter	4832732618637221	Sun2251

```
[10]: def quality(total_bill,tip):
      if tip/total_bill > 0.25:
          return 'Generous'
      else:
          return 'Other'
```

```
[12]: quality(16.99,1.01)
```

```
[12]: 'Other'
```

```
[14]: df['Quality'] = df[['total_bill','tip']].apply(lambda df: quality(df['total_bill'],df['tip']),axis=1)
```

```
[18]: df['Quality'] = np.vectorize(quality)(df['total_bill'],df['tip'])
df
```

	total_bill	tip	sex	smoker	day	time	size	price_per_person	Payer Name	CC Number	Payment ID	Quality
0	16.99	1.01	Female	No	Sun	Dinner	2	8.49	Christy Cunningham	3560325168603410	Sun2959	Other
1	10.34	1.66	Male	No	Sun	Dinner	3	3.45	Douglas Tucker	4478071379779230	Sun4608	Other
2	21.01	3.50	Male	No	Sun	Dinner	3	7.00	Travis Walters	6011812112971322	Sun4458	Other
3	23.68	3.31	Male	No	Sun	Dinner	2	11.84	Nathaniel Harris	4676137647685994	Sun5260	Other
4	24.59	3.61	Female	No	Sun	Dinner	4	6.15	Tonya Carter	4832732618637221	Sun2251	Other
...	...	...	...	...	...	...	...	...	...	...	...	...
239	29.03	5.92	Male	No	Sat	Dinner	3	9.68	Michael Avila	5296068606052842	Sat2657	Other
240	2718	2.00	Female	Yes	Sat	Dinner	2	13.59	Monica Sanders	3506806155565404	Sat1766	Other
241	22.67	2.00	Male	Yes	Sat	Dinner	2	11.34	Keith Wong	6011891618747196	Sat3880	Other
242	17.82	1.75	Male	No	Sat	Dinner	2	8.91	Dennis Dixon	4375220550950	Sat17	Other
243	18.78	3.00	Female	No	Thur	Dinner	2	9.39	Michelle Hardin	3511451626698139	Thur672	Other

244 rows x 12 columns

Refer Obsidian for notes on np.vectorize()